ZENCRO

SCHOOL MANAGEMENT SYSTEM

K–12 School Management Platform

BACKEND ↔ FRONTEND API & DATA CONTRACT SPECIFICATION

Practical development reference for parallel backend + frontend work

ZAINAB — Backend / Developer
MARIYA — Frontend / Developer

8-Day MVP Development Plan • Internal Use • English-only MVP

Source basis: finalized K–12 research, finalized MVP specification, and the supplied prototype/reference screens. This document translates those decisions into a build-oriented API/data contract; it does not replace the product documents.

1. Purpose & How to Use This Document

This is the shared working contract between backend and frontend. It answers: **what data must exist and what APIs expose it?** and **what does each screen send and receive?**

Zainab: use the table inventory, relationships, API cards, validation rules and RLS notes as the backend implementation checklist.

Mariya: use the screen → API → data mapping, response shapes and UI-state notes while building with mock data.

API/Data Contract Sync, simply: both developers agree on the exact field names, types and meaning of data crossing between them. If Mariya expects *first_name* and Zainab returns *firstName*, the contract is not synced.

2. Technical Baseline

Layer	MVP direction	Use
Frontend	Next.js + React + TypeScript + Tailwind + shadcn/ui	Screens, forms, tables, dashboards
Backend	Next.js Route Handlers / Server Actions	Business logic and API layer
Database	Supabase PostgreSQL	Core relational school data
Auth	Supabase Auth	Identity/session
Authorization	RBAC + Supabase RLS	Role + school isolation
Storage	Supabase Storage	Files/documents where required
Offline	IndexedDB + Dexie, only for validated MVP workflow	Attendance/offline candidate
Deployment	Vercel / agreed direction	Web deployment

Architecture principle: one main web application, one primary backend layer and Supabase. No microservices, queues, Kubernetes, Redis or other enterprise infrastructure is needed for this MVP.

3. Core Data Model

Table	Purpose	Important MVP fields	Main relationships
schools	School workspace / tenant	id, name, code, board, curriculum, contact, status	users, academic_years, classes, students
academic_years	Academic periods	id, school_id, name, start_date, end_date, is_current	schools, classes, exams, fees
classes	Grade/class	id, school_id, academic_year_id, name, level, profile_id	sections, students, subjects, report cards
sections	Class sections	id, class_id, name, class_teacher_id	students, attendance
subjects	Subject catalog	id, school_id, name, code, active	class_subjects, exams
class_subjects	Subject/teacher assignment	id, class_id, subject_id, teacher_id	classes, subjects, users
users	Staff/parent identity	id, auth_user_id, school_id, role, name, email, status	schools, assignments
students	Core student record	id, school_id, admission_no, first_name, last_name, dob, gender, class_id, section_id, status	guardians, attendance, fees, results
guardians	Parent/guardian	id, school_id, name, phone, email, relation	student_guardians
student_guardians	Student ↔ guardian	student_id, guardian_id, is_primary	students, guardians
attendance_records	Daily attendance	id, school_id, student_id, date, status, marked_by, marked_at, sync_status	students, users
fee_structures	Fee heads/amounts	id, school_id, academic_year_id, name, amount, due_date	student_fee_items
student_fee_items	Student fee balance	id, student_id, fee_structure_id, amount_due, amount_paid, status	students, fee_structures, payments
fee_payments	Payment/receipt	id, student_fee_item_id, receipt_no, amount, payment_date, method, reference, recorded_by	fee items, users
exams	Exam period	id, school_id, academic_year_id, name, start_date, end_date, status	exam_subjects, results
exam_subjects	Exam subject/max marks	id, exam_id, subject_id, max_marks, pass_marks	exams, subjects, results
results	Student marks/grade	id, student_id, exam_subject_id, marks, grade, remarks, status	students, exams, subjects
report_card_profiles	Bounded report-card config	id, school_id, board, academic_level, name, template_config, active	classes
class_report_card_profiles	Class → profile mapping	class_id, profile_id	classes, profiles
report_cards	Generated snapshot	id, student_id, profile_id, academic_year_id, generated_at, status, snapshot_data	students, profiles
announcements	School/class communication	id, school_id, title, body, audience_type, audience_ids, published_at, created_by, status	users, classes
ptm_meetings	PTM event	id, school_id, title, date, start_time, end_time, class_id, status	ptm_slots
ptm_slots	Bookable slots	id, meeting_id, teacher_id, start_time, end_time, capacity	ptm_bookings
ptm_bookings	Parent booking	id, slot_id, student_id, parent_id, status, booked_at	students, guardians/users
forms	Basic digital form	id, school_id, title, description, form_type, fields_json, due_date, status, created_by	form_responses
form_responses	Submitted response	id, form_id, student_id, submitted_by, response_json, submitted_at, status	forms, students
import_jobs	Import/migration job	id, school_id, type, file_name, status, counts, created_by, created_at	import_errors
import_errors	Validation issues	id, job_id, row_no, field, message, severity	import_jobs
export_jobs	Export job/history	id, school_id, type, format, filters_json, status, created_by, created_at, file_path	users
audit_logs	Activity/audit information	id, school_id, user_id, action, entity_type, entity_id, created_at, metadata	users, school

Technical recommendation: do not create tables for dashboard cards or summary numbers. Compose those from operational data.

4. Main Table Relationships

Core: School → Academic Year → Class → Section → Student

Academic: Class → Class Subjects → Subject / Teacher

Student 360: Student → Guardians; Attendance; Fees → Payments; Results → Exams/Subjects; Forms; PTM

Report cards: Board/Curriculum + Academic Level → Profile → Class Assignment → Results → Generated Report Card

Communication: Announcement → Audience; Guardian ↔ Student → Parent Dashboard; PTM → Slots → Booking

Data movement: CSV/Excel → Import Job → Validate → Preview → Confirm → Core Tables → Reconcile; Core Tables → Export Job

Tenant rule: every school-owned record must be traceable to a school, and access must be constrained by authenticated user role plus school scope.

5. Role / Permission Reference

Module	Admin	Teacher	Accountant	Parent
School Setup	Full	—	—	—
Students / Student 360	Full	Assigned/limited	View as needed	Own children
Attendance	Manage	Assigned classes	—	Own children view
Fees	Manage	View if needed	Manage	Own children view
Exams / Results	Manage	Assigned subjects/classes	—	Own children view
Communication	Publish/manage	If permitted	—	Receive
Parent Dashboard	—	—	—	Own children
PTM	Manage	Own slots	—	Book/view own
Forms	Manage	Assigned	—	Respond
Import / Export	Manage	—	Where permitted	—
Data & Privacy	Manage	Own access	Own access	Own access

DAY 1 Foundation

Day goal: Make database/API conventions and the frontend shell agree before feature work.

Zainab: Supabase connection, migrations, core user/role/school foundation, API conventions, date/status conventions.

Mariya: app shell, navigation, role-aware placeholders, shared loading/empty/error states.

Sync: agree field naming, pagination, dates, status values, error shape and mock objects.

GET	/api/me
Purpose	Return authenticated identity, role and school context.
Request / Input	No body.
Response / Output	{ user:{ id,name,email,role,school_id }, school:{ id,name } }
Frontend usage	Render role-aware shell.
Backend responsibility	Validate session; derive school from auth.
Permission / RLS	Authenticated user

DAY 2 Auth + School Setup

Day goal: Create the minimum school structure every later module depends on.

GET /api/school

Purpose	Fetch current school setup.
Request / Input	No body.
Response / Output	{ school, current_academic_year }
Frontend usage	School Setup/header.
Backend responsibility	School scope.
Permission / RLS	Admin

PATCH /api/school

Purpose	Update school basics.
Request / Input	{ name, code, board, curriculum, contact }
Response / Output	{ school }
Frontend usage	Save setup form.
Backend responsibility	Validate allowed fields.
Permission / RLS	Admin

GET /api/academic-years

Purpose	List academic years.
Request / Input	?status=active|all
Response / Output	{ academic_years:[...] }
Frontend usage	Year selectors.
Backend responsibility	School scope.
Permission / RLS	Admin

POST /api/academic-years

Purpose	Create academic year.
Request / Input	{ name,start_date,end_date }
Response / Output	{ academic_year }
Frontend usage	Create year.
Backend responsibility	Valid dates; one current year.
Permission / RLS	Admin

GET /api/classes

Purpose	List classes/sections.
Request / Input	?academic_year_id
Response / Output	{ classes:[{id,name,level,sections:[...]}] }
Frontend usage	Setup selectors.
Backend responsibility	School scope.
Permission / RLS	Admin/Teacher

POST /api/classes

Purpose	Create class/section.
Request / Input	{ name, level, section_names:[] }
Response / Output	{ class, sections }
Frontend usage	Setup forms.
Backend responsibility	Unique within school/year.
Permission / RLS	Admin

GET /api/subjects

Purpose	List subjects.

Request / Input	?class_id
Response / Output	{ subjects:[...] }
Frontend usage	Subject selectors.
Backend responsibility	School scope.
Permission / RLS	Admin/Teacher

DAY 3 Students + Student 360

Day goal: Build the central student record and one aggregated Student 360 response.

Main tables: students, guardians, student_guardians, classes, sections, users.

Student 360 is an API view/response, not a giant student table. It combines basic student data with summaries from other modules.

GET	/api/students
Purpose	List/search/filter students.
Request / Input	?class_id&section;_id&search;&page;&page;_size
Response / Output	{ students:[{id,admission_no,first_name,last_name,class,section,status}], total }
Frontend usage	Student table.
Backend responsibility	School scope + pagination.
Permission / RLS	Admin/Teacher

POST	/api/students
Purpose	Create student and optional guardians.
Request / Input	{ admission_no,first_name,last_name,dob,gender,class_id,section_id,guardians:[...] }
Response / Output	{ student, guardians }
Frontend usage	Add Student form.
Backend responsibility	Unique admission_no; valid class/section.
Permission / RLS	Admin

PATCH	/api/students/:id
Purpose	Edit core student data.
Request / Input	Partial student fields.
Response / Output	{ student }
Frontend usage	Edit form.
Backend responsibility	School isolation.
Permission / RLS	Admin

GET	/api/students/:id
Purpose	Basic details + guardians.
Request / Input	Path id.
Response / Output	{ student, guardians }
Frontend usage	Profile screen.
Backend responsibility	School isolation.
Permission / RLS	Admin/Teacher/Parent*

GET	/api/students/:id/360
Purpose	Aggregated Student 360.
Request / Input	?academic_year_id
Response / Output	{ student,guardians,attendance_summary,fee_summary,academic_summary,forms,ptm,recent_activity }
Frontend usage	Student 360 tabs/cards.
Backend responsibility	Role-based field filtering.
Permission / RLS	Admin/Teacher/Parent-own-child

UI area	Primary source
Basic information	students + classes + sections
Guardian	guardians + student_guardians
Attendance	attendance_records
Fees	student_fee_items + fee_payments
Academic	results + exams + subjects
Forms	forms + form_responses

UI area	Primary source
PTM	ptm_bookings + ptm_meetings
Recent activity	audit_logs / selected operational events

DAY 4 Attendance + Fees

Day goal: Make attendance fast and fee information clear without building a full accounting system.

GET /api/attendance

Purpose	Load class/date attendance.
Request / Input	?class_id&section;_id&date;
Response / Output	{ date,class,section,students:[{student_id,name,status,attendance_percentage}] }
Frontend usage	Attendance grid.
Backend responsibility	One record/student/date.
Permission / RLS	Admin/Teacher

POST /api/attendance/bulk

Purpose	Save class/date attendance.
Request / Input	{ date,class_id,section_id,records:[{student_id,status}] }
Response / Output	{ saved_count,errors:[] }
Frontend usage	Save Attendance.
Backend responsibility	Reject duplicates; validate membership.
Permission / RLS	Admin/Teacher

GET /api/students/:id/attendance

Purpose	Attendance history/summary.
Request / Input	?from&to;
Response / Output	{ summary:{percentage,present,absent,late}, records:[...] }
Frontend usage	Student 360 / parent.
Backend responsibility	Own-child rule.
Permission / RLS	Admin/Teacher/Parent

GET /api/fees/summary

Purpose	Fee totals.
Request / Input	?academic_year_id
Response / Output	{ total_fees,collected,outstanding,collection_percentage }
Frontend usage	Fees/Admin cards.
Backend responsibility	Calculated from fee items/payments.
Permission / RLS	Admin/Accountant

GET /api/fees/students

Purpose	Student fee balances.
Request / Input	?class_id&section;_id&status;&search;
Response / Output	{ students:[{student_id,name,class,total_due,paid,pending,status}],total }
Frontend usage	Fees table.
Backend responsibility	Role-limited payment detail.
Permission / RLS	Admin/Accountant

POST /api/fees/payments

Purpose	Record payment.
Request / Input	{ student_id,fee_item_id,amount,payment_date,method,reference }
Response / Output	{ payment,updated_balance,receipt }
Frontend usage	Record Payment.
Backend responsibility	Positive amount; valid balance; unique receipt.
Permission / RLS	Admin/Accountant

GET /api/students/:id/fees

Purpose	Student fee details.

Request / Input	?academic_year_id
Response / Output	{ total_due,paid,pending,items:[...],payments:[...] }
Frontend usage	Student 360 / parent fees.
Backend responsibility	Own-child rule.
Permission / RLS	Admin/Accountant/Parent

Offline attendance: if enabled, local pending records must include sync status; backend must reject duplicates/conflicts deterministically. Offline launch validation remains TO BE CONFIRMED.

DAY 5 Exams + Results + Report Cards

Day goal: Store results cleanly and support bounded report-card configuration.

GET /api/exams

Purpose	List exams.
Request / Input	?academic_year_id&class;_id
Response / Output	{ exams:[...] }
Frontend usage	Exam selector.
Backend responsibility	Year/class scope.
Permission / RLS	Admin/Teacher

POST /api/exams

Purpose	Create exam.
Request / Input	{ name,academic_year_id,start_date,end_date }
Response / Output	{ exam }
Frontend usage	Create exam.
Backend responsibility	Valid dates.
Permission / RLS	Admin

GET /api/exams/:id/results

Purpose	Load marks/results.
Request / Input	?class_id&subject;_id
Response / Output	{ exam,subjects:[{subject_id,max_marks,results:[...]}] }
Frontend usage	Marks/results.
Backend responsibility	Teacher assignment scope.
Permission / RLS	Admin/Teacher

POST /api/results/bulk

Purpose	Save marks/results.
Request / Input	{ exam_subject_id,records:[{student_id,marks,grade?,remarks?}] }
Response / Output	{ saved_count,errors:[] }
Frontend usage	Enter Marks.
Backend responsibility	Marks <= max; correct class.
Permission / RLS	Admin/Teacher

GET /api/report-card-profiles

Purpose	List report-card profiles.
Request / Input	?board&academic;_level
Response / Output	{ profiles:[{id,name,board,academic_level,template_config}] }
Frontend usage	Config/template selector.
Backend responsibility	Admin config only.
Permission / RLS	Admin

POST /api/report-card-profiles

Purpose	Create/update bounded configuration.
Request / Input	{ board,academic_level,name,template_config }
Response / Output	{ profile }
Frontend usage	Configuration form.
Backend responsibility	Validate supported keys.
Permission / RLS	Admin

POST /api/report-card-profiles/:id/classes

Purpose	Assign profile to classes.

Request / Input	{ class_ids:[] }
Response / Output	{ assignments:[...] }
Frontend usage	Class mapping.
Backend responsibility	No duplicate active mapping.
Permission / RLS	Admin

GET **/api/report-cards/:studentId**

Purpose	Return report-card preview data.
Request / Input	?academic_year_id&profile;_id
Response / Output	{ student,school,profile,subjects:[...],attendance?,overall,generated_at }
Frontend usage	Preview.
Backend responsibility	Same-school student/profile.
Permission / RLS	Admin/Teacher/Parent*

POST **/api/report-cards/:studentId/generate**

Purpose	Persist generated snapshot.
Request / Input	{ academic_year_id,profile_id }
Response / Output	{ report_card_id,status,snapshot_data }
Frontend usage	Generate/download.
Backend responsibility	Deterministic results + config.
Permission / RLS	Admin

Recommended bounded template configuration: sections such as student header, subject results, attendance, overall, marks/grade display, remarks and school branding. The exact supported keys are a TECHNICAL RECOMMENDATION; do not build a free-form Canva-style editor.

DAY 6 Communication + Parent + PTM + Forms

Day goal: Connect school-to-family communication and simple digital workflows.

GET /api/announcements

Purpose	List visible announcements.
Request / Input	?audience&page;
Response / Output	{ announcements:[...] }
Frontend usage	Communication list.
Backend responsibility	Filter audience server-side.
Permission / RLS	Admin/Teacher/Parent

POST /api/announcements

Purpose	Create/publish announcement.
Request / Input	{ title,body,audience_type,audience_ids?,publish_at? }
Response / Output	{ announcement }
Frontend usage	Create announcement.
Backend responsibility	Allowed publisher + valid audience.
Permission / RLS	Admin / permitted Teacher

GET /api/parent/dashboard

Purpose	Load parent home for linked children.
Request / Input	No body; auth identifies parent.
Response / Output	{ parent,children:[...],announcements,upcoming_ptm,forms }
Frontend usage	Parent mobile dashboard.
Backend responsibility	Only linked children.
Permission / RLS	Parent

GET /api/ptm/meetings

Purpose	List PTMs and slots.
Request / Input	?class_id&date;
Response / Output	{ meetings:[{...,slots:[...]}] }
Frontend usage	PTM screen.
Backend responsibility	School/class scope.
Permission / RLS	Admin/Teacher/Parent

POST /api/ptm/bookings

Purpose	Book PTM slot.
Request / Input	{ slot_id,student_id }
Response / Output	{ booking }
Frontend usage	Book Slot.
Backend responsibility	Reject full/already-booked; verify parent-child.
Permission / RLS	Parent

POST /api/ptm/slots

Purpose	Create/update slot.
Request / Input	{ meeting_id,teacher_id,start_time,end_time,capacity }
Response / Output	{ slot }
Frontend usage	PTM setup.
Backend responsibility	No invalid overlap if rule enabled.
Permission / RLS	Admin/Teacher

GET /api/forms

Purpose	List relevant forms.

Request / Input	?status
Response / Output	{ forms:[...] }
Frontend usage	Forms list.
Backend responsibility	Audience/child scope.
Permission / RLS	Admin/Teacher/Parent

POST **/api/forms**

Purpose	Create basic form.
Request / Input	{ title,description,form_type,fields,due_date }
Response / Output	{ form }
Frontend usage	Create form.
Backend responsibility	Supported field types only.
Permission / RLS	Admin

POST **/api/forms/:id/responses**

Purpose	Submit response.
Request / Input	{ student_id,response }
Response / Output	{ response,submitted_at }
Frontend usage	Parent/teacher response.
Backend responsibility	Prevent duplicates per rule.
Permission / RLS	Parent/Teacher

DAY 7 Import + Export + Dashboards + Data / Privacy

Day goal: Finish operational data management and make the product integration-ready.

POST /api/imports

Purpose	Start import/upload metadata.
Request / Input	{ type,file_name,mapping }
Response / Output	{ import_id,status:'uploaded' }
Frontend usage	Import first step.
Backend responsibility	File type/size + school scope.
Permission / RLS	Admin

POST /api/imports/:id/validate

Purpose	Validate upload.
Request / Input	Mapping if needed.
Response / Output	{ valid_count,error_count,warning_count,errors:[...] }
Frontend usage	Validation/review.
Backend responsibility	No core writes before confirmation.
Permission / RLS	Admin

GET /api/imports/:id

Purpose	Get import status/details.
Request / Input	Path id.
Response / Output	{ job,errors,preview }
Frontend usage	History/review.
Backend responsibility	School scope.
Permission / RLS	Admin

POST /api/imports/:id/confirm

Purpose	Commit validated import.
Request / Input	{ confirm:true }
Response / Output	{ imported_count,skipped_count,errors:[] }
Frontend usage	Confirm Import.
Backend responsibility	Controlled writes + audit.
Permission / RLS	Admin

GET /api/exports

Purpose	List export history.
Request / Input	?type&status;&page;
Response / Output	{ exports:[...] }
Frontend usage	Export history.
Backend responsibility	School/data scope.
Permission / RLS	Admin/authorized

POST /api/exports

Purpose	Create export job.
Request / Input	{ type,format,filters }
Response / Output	{ export_id,status }
Frontend usage	Data Export.
Backend responsibility	Role + data scope; log export.
Permission / RLS	Admin/authorized

GET /api/dashboard/admin

Purpose	Action-oriented admin dashboard.

Request / Input	?date
Response / Output	{ priorities,student_overview,fees,attendance,activity,upcoming_ptm,communication,data_status }
Frontend usage	Admin dashboard.
Backend responsibility	Return compact summaries.
Permission / RLS	Admin

GET **/api/dashboard/teacher**

Purpose	Today-focused teacher dashboard.
Request / Input	?date
Response / Output	{ teacher,classes,today_tasks,attendance,academic_work,ptm,communication,forms }
Frontend usage	Teacher dashboard.
Backend responsibility	Assigned scope only.
Permission / RLS	Teacher

GET **/api/data-privacy/summary**

Purpose	Data/privacy center summary.
Request / Input	No body.
Response / Output	{ roles,access_summary,recent_activity,exports,retention_info }
Frontend usage	Data & Privacy.
Backend responsibility	Avoid exposing unnecessary private data.
Permission / RLS	Admin/authorized

Privacy guardrail: Auth/RLS/audit controls are technical safeguards; they are not a claim of DPDP compliance.

DAY 8 Full Integration + Testing + Security + Deployment

Day goal: Replace mocks with real APIs and verify end-to-end workflows.

Area	Test
Auth	Login/session/logout; unauthorized requests rejected
RBAC	Role restrictions work for Admin, Teacher, Accountant and Parent
RLS / tenant isolation	School A cannot read/write School B data
Students	Create → edit → Student 360 → guardian link
Attendance	Mark → save → correct → percentage/history → offline conflicts if enabled
Fees	Assign → payment → balance → receipt → dashboard summary
Exams	Create → marks → results → report-card preview
Report cards	Profile mapping → configuration → preview → snapshot
Communication	Publish → correct audience visibility
Parent	Multi-child → attendance/fees/results/forms/PTM
PTM	Meeting → slots → booking → double-booking rejection
Forms	Create → target → submit → status
Import/export	Validate before write → confirm → reconcile → export → history
Dashboards	Real API summaries match underlying records
UI states	Loading, empty, validation and API errors
Security	Server validation, no trusted client school_id, no exposed secrets
Deployment	Environment variables, migrations, build, smoke test

13. Master API Inventory

Day	Module	Method	Endpoint	Frontend	Role
1	Foundation	GET	/api/me	App shell	Authenticated
2	School	GET/PATCH	/api/school	School Setup	Admin
2	Academic	GET/POST	/api/academic-years	Academic Year	Admin
2	Classes	GET/POST	/api/classes	Class/Section Setup	Admin
2	Subjects	GET	/api/subjects	Subject Setup	Admin/Teacher
3	Students	GET	/api/students	Student List	Admin/Teacher
3	Students	POST	/api/students	Add Student	Admin
3	Students	PATCH	/api/students/:id	Edit Student	Admin
3	Student 360	GET	/api/students/:id/360	Student 360	Admin/Teacher/Parent
4	Attendance	GET/POST	/api/attendance*	Attendance	Admin/Teacher
4	Fees	GET	/api/fees/summary	Fees Dashboard	Admin/Accountant
4	Fees	GET/POST	/api/fees/students; /api/fees/payments	Fees	Admin/Accountant
4	Student Fees	GET	/api/students/:id/fees	Student 360/Parent	Admin/Accountant/Parent
5	Exams	GET/POST	/api/exams*	Exams	Admin/Teacher
5	Results	GET/POST	/api/exams/:id/results; /api/results/bulk	Results	Admin/Teacher
5	Report Cards	GET/POST	/api/report-card-profiles*	Configuration	Admin
5	Report Cards	GET/POST	/api/report-cards/:studentId*	Preview/Generate	Admin/Teacher/Parent*
6	Communication	GET/POST	/api/announcements*	Communication	Admin/Teacher/Parent
6	Parent	GET	/api/parent/dashboard	Parent Dashboard	Parent
6	PTM	GET/POST	/api/ptm/*	PTM	Admin/Teacher/Parent
6	Forms	GET/POST	/api/forms*	Forms	Admin/Teacher/Parent
7	Import	POST/GET	/api/imports*	Import/Migration	Admin
7	Export	POST/GET	/api/exports*	Data Export	Admin/authorized
7	Dashboards	GET	/api/dashboard/admin	Admin Dashboard	Admin
7	Dashboards	GET	/api/dashboard/teacher	Teacher Dashboard	Teacher
7	Privacy	GET	/api/data-privacy/summary	Data & Privacy	Admin/authorized

* Exact sub-route naming versus server actions is a technical implementation choice. The contract matters more than the transport style.

14. Master Database / Table Inventory

Table	Purpose	Module
schools	Tenant/school workspace	Foundation
academic_years	Academic periods	School Setup
users	Identity + role/school link	Auth/RBAC
classes	Grade/class	School Setup
sections	Class sections	School Setup
subjects	Subject catalog	School Setup/Exams
class_subjects	Subject/teacher assignment	School Setup/Teacher
students	Core student	Students/360
guardians	Parent/guardian	Students/Parent
student_guardians	Student ↔ guardian	Students/Parent
attendance_records	Daily attendance	Attendance
fee_structures	Fee heads/amounts	Fees
student_fee_items	Student balances	Fees
fee_payments	Payments/receipts	Fees
exams	Exam definitions	Exams
exam_subjects	Exam subjects/max marks	Exams
results	Marks/grades	Results
report_card_profiles	Bounded config	Report Cards
class_report_card_profiles	Class → profile	Report Cards
report_cards	Generated snapshot	Report Cards
announcements	Communication	Communication
ptm_meetings	PTM events	PTM
ptm_slots	Bookable slots	PTM
ptm_bookings	Parent bookings	PTM
forms	Form definitions	Forms
form_responses	Responses	Forms
import_jobs	Import/migration jobs	Data Management
import_errors	Validation issues	Data Management
export_jobs	Export jobs/history	Data Management
audit_logs	Activity/audit information	Data/Privacy

Do not add tables just for UI cards. If a value is a summary, calculate it from operational data or return it through a summary endpoint.

15. Frontend ↔ Backend Dependency Map

Frontend screen	API	Primary data	UI output
Login / app shell	Auth + /api/me	users, schools	Identity, role, school
School Setup	/api/school; /api/academic-years; /api/classes; /api/subjects	schools, academic_years, classes, sections, subjects	Setup forms/selectors
Student List	/api/students	students, classes, sections	Search/filter table
Student 360	/api/students/:id/360	students + guardians + attendance + fees + results + forms + PTM	Cards + tabs
Attendance	/api/attendance*	attendance_records	Class/date grid + summary
Fees	/api/fees/summary; /api/fees/students; /api/fees/payments	fee structures/items/payments	Totals + balances + payment
Exams & Results	/api/exams*; /api/results*	exams + subjects + results	Marks/result table
Report Card Config	/api/report-card-profiles*	profiles + class mapping	Configuration
Report Card Preview	/api/report-cards/:studentId	results + profile + student	Preview
Communication	/api/announcements*	announcements	Announcement list
Parent Dashboard	/api/parent/dashboard	linked children + summaries	Multi-child cards
PTM	/api/ptm/*	meetings + slots + bookings	Booking UI
Forms	/api/forms*	forms + responses	Pending/submitted
Import/Migration	/api/imports*	jobs + errors + target tables	Validate → preview → confirm
Data Export	/api/exports*	export_jobs + target tables	Format/filter/history
Admin Dashboard	/api/dashboard/admin	operational summaries	Priorities + quick actions
Teacher Dashboard	/api/dashboard/teacher	assignments + tasks	Today's tasks
Data & Privacy	/api/data-privacy/summary	users + audit + exports	Access/activity/privacy

16. API / Data Contract Sync Checklist

Check	Question
■ Endpoint name	Do both developers use the same name?
■ Request fields	What exactly does the frontend send?
■ Response fields	What exactly does the backend return?
■ Field names	Are names identical in the contract?
■ Data types	String, number, boolean, date, array, object?
■ Required/optional	Which fields may be null/absent?
■ Validation	What is rejected and why?
■ Permission	Which role may read/write?
■ Error shape	Can the frontend display the error?
■ Loading/empty state	What does the UI show while/no data?
■ Mock data	Does mock data match the real response?
■ RLS/tenant	Can the user access only the correct school?
■ Success condition	What does the UI consider a successful save/fetch?

Example: Mariya expects *admission_no*; Zainab returns *admissionNumber*. That is a contract mismatch. Catch it during sync, not on Day 8.

17. Recommended Error Response Standard

This is a **TECHNICAL RECOMMENDATION**.

```
{"success": false, "error": {"code": "VALIDATION_ERROR", "message": "Admission number already exists.", "field": "admission_no"}}
```

Suggested codes: AUTH_REQUIRED, FORBIDDEN, NOT_FOUND, VALIDATION_ERROR, CONFLICT, INTERNAL_ERROR. Use *field* for form-level errors where possible.

18. Mock Data → Real API Integration Strategy

Phase	Mariya	Zainab
Days 1–7	Use mock objects matching the agreed response shape	Build DB/API against the same contract
Contract change	Update mock + UI expectations	Update API + document change before proceeding
Day 8	Switch mock services to real calls	Verify real responses, permissions and RLS
Bug fixing	UI/state/validation fixes	API/data/permission fixes

19. Final 8-Day Development Checklist

Day	Backend / Zainab	Frontend / Mariya	Exit condition
1	Foundation, migrations, conventions	App shell + shared states	Names/types/error contract agreed
2	Auth, RBAC/RLS, school setup	Login + setup screens	School/class/subject contracts stable
3	Students + Student 360 APIs	Student list/add/edit/360	360 response stable
4	Attendance + Fees APIs	Attendance + fees UI	Daily workflows match contract
5	Exams/results/report-card APIs	Marks/results/config/preview	Report-card model fixed
6	Communication/parent/PTM/forms APIs	Parent/PTM/forms/communication UI	Audience/booking/response contracts stable
7	Import/export/dashboards/privacy APIs	Management/dashboard/privacy UI	Feature freeze / integration-ready
8	Integration/security/bugs/deployment	Real API integration + responsive/error testing	End-to-end workflows pass

20. Scope Guardrails & Open Decisions

Outside MVP: transport, library, hostel, HR/payroll, inventory, advanced accounting, full LMS, advanced timetable engine, AI chatbot, predictive analytics, native mobile apps, college/university management, automatic implementation of every board's rules, automatic migration from every ERP.

Open decisions carried forward: attendance exception statuses/correction/offline validation; student status/document list; teacher-day definition without an advanced timetable engine; report-card parent visibility and exact bounded options; announcement publishing roles; exact fee reconciliation views; import record types at launch; activity/retention depth; PTM slot rules.

Final principle: keep the connected core simple, deterministic, testable and easy for two developers to build in parallel.